

Relatório do Trabalho Prático OpenMP

Nome: Diógenes Antonio Falcade Pereira

A função LCS tem como objetivo calcular o comprimento da maior subsequência comum entre duas cadeias de caracteres, utilizando para isso uma matriz que armazena os valores parciais dessa contagem. A matriz é iniciada com zeros na primeira linha e na primeira coluna para garantir que a primeira comparação tenha sempre um valor anterior válido. O funcionamento da lógica é simples: o loop externo percorre a sequência B e o interno compara com cada caractere da sequência A. Se os caracteres forem iguais, soma-se 1 ao valor da diagonal anterior. Se forem diferentes, é escolhido o maior valor entre o elemento de cima e o da esquerda, garantindo que a contagem nunca diminua. Essa estrutura reflete o fato de que uma subsequência vai crescendo à medida que os caracteres coincidem. Como esperado, a linha 1 terá vários valores iguais a 1 a partir da primeira ocorrência de coincidência, e não passa disso, já que a base era zero. Esse padrão se repete pela matriz e, ao final, o maior score estará armazenado no canto inferior direito, indicando o tamanho da subsequência comum encontrada. A coluna 1 segue o mesmo comportamento, com os incrementos se espalhando para baixo e para a direita à medida que as comparações vão acontecendo.

A abordagem baseada em blocos, conhecida como Blocked Dynamic Programming (BDP), foi a que apresentou melhor desempenho entre as estratégias testadas. A lógica é dividir a matriz de pontuação em blocos menores de tamanho fixo (ou tiles), que serão processados de forma independente, desde que suas dependências de dados estejam satisfeitas. Em vez de percorrer célula a célula, ou linha a linha como na versão sequencial, cada tarefa agora representa um bloco inteiro da matriz, permitindo que múltiplos blocos sejam processados em paralelo sempre que possível.

Na prática, o algoritmo calcula quantos blocos serão necessários tanto na horizontal (em relação à sequência A) quanto na vertical (em relação à sequência B), com base no TILE_SIZE definido. Para cada combinação de bloco vertical e horizontal (by, bx), é criada uma tarefa (`#pragma omp task`) com a diretiva `depend`, que especifica quais blocos devem ser processados antes daquele atual. No caso da LCS, cada bloco depende diretamente do bloco acima e do bloco à esquerda, além do bloco da diagonal superior esquerda, por causa da estrutura do algoritmo original. A área interna do bloco é percorrida com dois laços `for`, exatamente como na versão sequencial, com a diferença de que os limites são locais ao bloco, e não globais à matriz. Isso mantém a lógica original do algoritmo LCS preservada dentro de cada bloco. A vantagem é que a paralelização ocorre no nível de blocos inteiros, o que reduz drasticamente o overhead de gerenciamento de tarefas.

Outro ponto importante dessa abordagem é o aproveitamento do cache. Como os blocos trabalham com regiões contíguas da matriz, os dados acessados tendem a permanecer nas hierarquias mais rápidas de memória, reduzindo o número de acessos à RAM. Esse comportamento melhora ainda mais o desempenho, especialmente em matrizes grandes. O fato de cada thread processar um bloco completo também evita problemas de falsos compartilhamentos e melhora a localidade espacial dos dados. Cada thread pega um bloco assim que outra termina, e como as dependências são resolvidas automaticamente pelo compilador com base nas diretivas `depend`, o escalonamento acontece de forma segura.

Algo bastante aproveitado foi o uso da cache, visto que processador utilizado (Ryzen 7 5700X) possui um cache L1 de 32 KB por núcleo, e um tile de 128x128 elementos mtype (2 bytes cada) ocupa cerca de 32 KB, que é justamente o tamanho do cache L1 de cada núcleo. No fim, essa abordagem aproveita muito bem os recursos disponíveis da máquina de testes.

O ambiente de testes tinha as seguintes características:

- Sistema Operacional: Ubuntu 24.04.2 LTS
- Kernel: Linux 6.11.0-25-generic
- Processador: AMD Ryzen 7 5700X (8 núcleos físicos, 16 threads)
- Compilador: GCC 13.3.0
- Flags de Compilação: `-O3 -ftree-vectorize` (otimização agressiva e vetorização automática)

Para evitar variações dinâmicas de clock e garantir desempenho máximo ativação do modo performance com `sudo cpufreq-set -g performance` forçando a CPU a operar na frequência máxima continuamente. Assim como Desativação do Turbo Boost com `echo "0" | sudo tee /sys/devices/system/cpu/cpufreq/boost` eliminando variações de clock devido a boost automático.

Para reduzir o impacto de processos em segundo plano utilizei `sudo renice 19 -p $(pgrep -f "background_process")` definindo prioridade mínima para processos não essenciais. E também limitação de recursos via cgroups que restringe uso da CPU por outros programas.

```
sudo cgcreate -g cpu:/experimento
sudo cgset -r cpu.shares=512 experimento # Limita alocação de CPU
sudo cgexec -g cpu:experimento ./meu_programa
```

Para otimizar o escalonamento de threads defini as variáveis de ambiente openMP

```
export OMP_PROC_BIND=close # Mantém threads próximas fisicamente
export OMP_PLACES=cores    # Aloca threads em núcleos físicos
```

```
export OMP_SCHEDULE=dynamic          # Escalonamento dinâmico de tarefas
```

Uso do perfstat para 40 execuções e coleta de dados de tempo médio e desvio padrão com o comando "perf stat -r 40 -e cycles,instructions,cache-references,cache-misses ./lcs"

A execução sequencial sem nenhuma flag leva aproximadamente 11,52062 +- 0,00486 segundos (+- 0.04%) com uma entrada de sizeA de 1.700.000 e sizeB 1500. Apenas como teste utilizei a flag de compilação O3 e tive um desempenho bem melhor ainda no algoritmo sequencial, que foi de 7,03240 +- 0,02304 (+- 0,32%). Outras não tiveram desempenho similar. Seguindo com a comparação, fiz a medição dos tempos de cada uma das funções que não paralelizei, que foram a leitura das sequências (read_seq e strlen), alocação e inicialização da matriz (allocateScoreMatrix e initScoreMatrix) e por fim a liberação de memória (freeScoreMatrix e free). Como termo de comparação, utilizei o pior caso e o melhor caso da execução sequencial para comparar quanto representa para ambas as execuções

	Leitura	Alocação	Liberação de memória	Total
Média (s)	0.0126711	0.00912045	0.36568925	0.3874808
Desvio padrão (s)	0.0004837	0.00085385	0.00222168	0.0023806
Porcentagem (%)	0.1099921	0.07917057	3.17438585	3.3635486
Porcentagem -O3 (%)	0.1801817	0.12969184	5.20006307	5.5099366

Considerando a equação da Lei de Amdahl, em que β é a fração da computação que só pode ser realizada sequencialmente e p o número de processadores.

$$S(p) = \frac{1}{\beta + \frac{(1-\beta)}{p}}$$

Para obter o speedup teórico da lei de Amdahl, é necessário definir qual é o tempo puramente sequencial. Dado que a solução que implementei paraleliza apenas a função do LCS, o tempo puramente sequencial é tudo aquilo que está fora da LCS. No programa sequencial e no paralelo utilizei clock_gettime entre as funções para calcular o tempo em segundos que as diversas funções, exceto LCS, levavam para executar. Para ser fiel a isso, capturei o tempo em 4 locais, ao iniciar o programa, antes do LCS, depois

do LCS e ao fim. Isso deve ser suficiente para calcular a quantidade de tempo sequencial, calculando o tempo total menos o tempo do LCS. Isso foi bem exemplificado na tabela acima. Portanto, utilizando os mesmos dados, considere o caso de 11.52 sequencial, que é executar o programa com o compilador normal. Porém, também vou usar o caso utilizando a flag de compilação O3, pois ela melhora de maneira significativa isso. Levantando ainda o fato de que o ganho posterior de paralelização não seria obtido com a paralelização em si, mas sim com a otimização da compilação que definitivamente faz o programa rodar mais rápido.

Tempo total (s)	2	4	8	∞
11.520	1.93491	3.63336	6.47538	29.73050
7.032	1.84012	3.17297	4.97459	18.14902

Vemos que com a otimização agressiva o speedup diminui, justamente pela velocidade adquirida através da compilação. Entretanto, nota-se também que o speedup não é tão expressivo com mais núcleos de processamento, pois mesmo que seja fornecida uma quantidade infinita, a região sequencial não poderá ser diminuída. De fato algumas funções também podiam ser paralelizadas com a diretiva OMP para loops for, mas rodando alguns testes não se mostrou um grande ganho, provavelmente porque o overhead de separar em 16 threads para uma atividade que já tomava pouco tempo era igualmente custoso a simplesmente deixar esse trecho sequencial.

Em resumo, utilizei entradas de tamanhos distintos para que o programa sequencial ficasse com maior tempo, variando tanto a entrada de A quanto a da sequência, como orientado. Utilizei sequências aleatórias de caracteres. Optei por usar um tamanho de A maior que B pensando no fato que o LCS é utilizado para encontrar subsequências em sequências extensas.

O número de threads é baseado no processador de 8 núcleos físicos e 16 threads. Abaixo, a tabela que resume os tamanhos das sequências comparadas, os tempos de cada execução e o desvio padrão para cada número de threads. Como mencionado anteriormente, foram executadas 40 vezes cada comparação para obter as métricas.

sizeA	sizeB	1	2	4	8	16
1700000	1500	7.05871 +- 0.00492	3.96650 +- 0.00250	2.26640 +- 0.00234	1.44390 +- 0.00412	1.28292 +- 0.00314
2000000	1600	9.4075 +- 0.0127	4.98720 +- 0.00393	2.86179 +- 0.00502	1.82957 +- 0.00549	1.62872 +- 0.00660

2500000	1700	12.7026 +/- 0.0172	6.72267 +/- 0.00521	3.83005 +/- 0.00483	2.40335 +/- 0.00530	2.05347 +/- 0.00741
---------	------	-----------------------	------------------------	------------------------	------------------------	------------------------

A eficiência pode ser obtida pela razão entre o tempo de execução paralelo e a multiplicação do tempo de execução sequencial pelo número de threads. Note que a eficiência aumenta gradativamente conforme aumentamos o tamanho das entradas. Isso será discutido posteriormente.

sizeA	sizeB	1	2	4	8	16
1700000	1500	1	0.88979	0.77862	0.61108	0.34387
2000000	1600	1	0.94316	0.82181	0.64273	0.36100
2500000	1700	1	0.94475	0.82914	0.66067	0.38661

Por fim, a tabela de speedup, que mostra que o aumento de velocidade é ligeiramente inferior ao speedup teórico, mas mesmo assim tem um desempenho bem satisfatório.

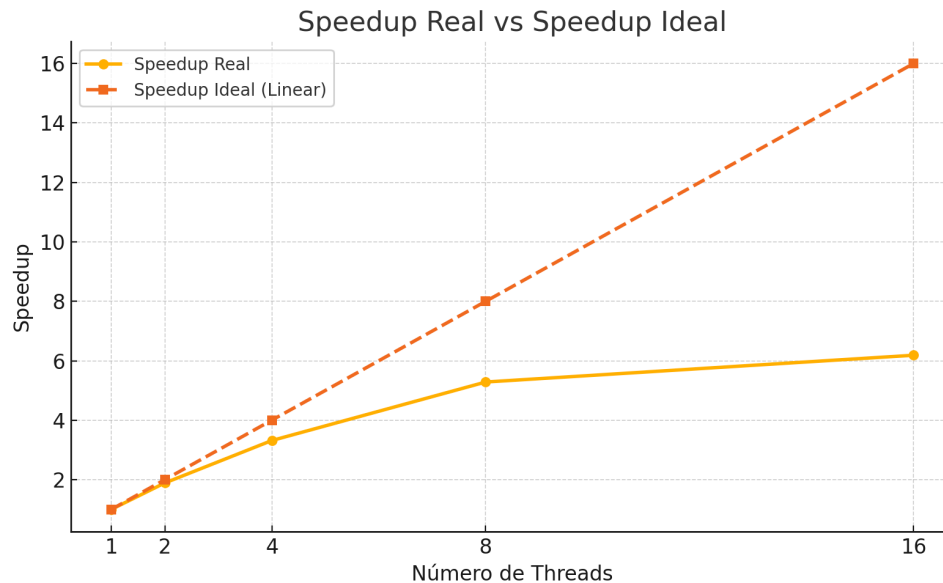
sizeA	sizeB	1	2	4	8	16
1700000	1500	1	1.77958	3.11450	4.88864	5.50206
2000000	1600	1	1.88632	3.28727	5.14191	5.77600
2500000	1700	1	1.88951	3.31656	5.28537	6.18591

Com base nos resultados obtidos nas execuções do algoritmo LCS paralelo, a ideia aqui é justamente entender se o algoritmo escalará bem com o aumento de recursos computacionais e qual o comportamento real frente ao esperado teoricamente, como por exemplo o speedup linear ou mesmo o previsto pela Lei de Amdahl.

Ao observar a tabela de speedup, vemos que os valores aumentam à medida que o número de threads cresce, o que já era esperado. No entanto, esse crescimento não é exatamente linear, mas isso é natural, pois conforme mais threads são alocadas, a parte sequencial do código se torna um limitador mais evidente. Ainda assim, os valores obtidos foram bastante satisfatórios. No melhor caso, com tamanho de entrada sizeA = 2500000 e sizeB = 1700, o speedup alcançado com 16 threads foi de aproximadamente 6,19x, valor que se aproxima da estimativa teórica da Lei de Amdahl considerando uma fração paralelizável em torno de 95%, como evidenciado nos testes anteriores.

Se compararmos esse speedup ao ideal (linear), ou seja, 16 threads entregando 16x de velocidade, vemos que o resultado real fica abaixo. A Lei de Amdahl deixa claro que, independentemente de quantos processadores sejam usados, a fração sequencial da

execução será sempre um limitador. Mesmo com otimizações como uso do compilador com flag `-O3`, o tempo puramente sequencial continua existindo e, portanto, limita o ganho. Vale destacar que nesse caso, a própria otimização do compilador já reduz o tempo de execução sequencial sem necessidade de paralelização, o que por sua vez impacta o cálculo do speedup, pois ele diminui não por uma deficiência do paralelismo, mas porque a base de comparação (execução sequencial) se tornou mais rápida.



A tabela de eficiência também traz informações importantes. Podemos observar que a eficiência cai gradualmente à medida que aumentamos o número de threads. Isso acontece por dois motivos principais: primeiro, o overhead da criação e gerenciamento das threads se torna mais significativo em relação ao tempo útil de processamento; segundo, há limitação prática de escalabilidade por conta do tamanho do problema e da região que não pode ser paralelizada. Com 2 ou 4 threads, a eficiência ainda se mantém razoável, beirando os 80% a 90%. Com 16 threads, no entanto, a eficiência cai para cerca de 34% no pior caso — o que indica que nem todas as threads estão sendo plenamente aproveitadas e o ganho marginal ao adicionar mais núcleos se torna cada vez menor. Esse comportamento é típico de escalabilidade forte, onde aumentamos os recursos computacionais para resolver o mesmo problema mais rapidamente. O que se observa aqui é que, até certo ponto, o programa responde bem ao aumento de threads, mas esse ganho tende a se estabilizar e decair, o que caracteriza perda de eficiência natural de soluções paralelas quando já se atinge boa distribuição de carga com menos threads.

Dito isso, acredito que a solução apresentada tem um bom grau de escalabilidade, dentro do que se espera de um algoritmo com dependência estrita em seus dados (como é o caso da matriz de LCS). A escalabilidade pode ser considerada forte até o

ponto de 4 a 8 threads, onde o aumento de desempenho acompanha razoavelmente o aumento de recursos. A partir desse ponto, observa-se um comportamento de saturação, em que o paralelismo deixa de ser o principal gargalo e a fração sequencial ou o overhead de threads começa a dominar a execução. Portanto, a solução é escalável, mas com limitação prática e previsível, conforme indicado também pela Lei de Amdahl e pelos testes feitos.

Os resultados estão coerentes com o esperado. As tendências seguem de forma lógica, tanto em termos de crescimento de speedup quanto de queda de eficiência, e os dados demonstram claramente que a aplicação das boas práticas (como controle de afinidade de threads, uso de perf, ajustes finos com OMP e cuidados com o ambiente de execução) tiveram papel importante para garantir a confiabilidade dos testes. O comportamento do programa mostra que ele se adapta bem ao aumento do problema, e que os ganhos de paralelização só se tornam realmente expressivos quando se trabalha com matrizes grandes o suficiente para compensar o overhead do paralelismo.